

Nombre:						Turno:										
Calificación:							+					+			=	
Importante - La duración del examen es de 2 horas y 40 minutos, a partir de la entrega del ejercicio. - Está terminantemente prohibido sacar del aula el examen o una copia de este. - Sólo se corregirán las preguntas escritas con bolígrafo. - Excepto las preguntas tipo test, desarrolle cada pregunta en hoja separada. - Comentar adecuadamente el código o pseudo-código, así como las variables utilizadas.																

1. Preguntas de tipo test (Cada respuesta correcta tiene una puntuación de 0'2. Por cada respuesta incorrecta se descontará 0'1. Sólo serán válidas las respuestas marcadas dentro de las casillas)

1.1.- ¿Es posible cambiar el tipo de una variable en C++?

- ☐ No. Nunca.
- ☐ Se puede cambiar usando typecasting.
- ☐ Solamente aquellas variables que no tengan un tipo.
- ☐ Sí. Mediante el operador typeid.

1.2.- ¿En C++ es posible definir excepciones personalizadas?

- ☐ Sí, para gestionar errores en tiempo de compilación.
- ☐ Sí, para gestionar tanto errores en tiempo de compilación como en tiempo de ejecución.
- ☐ Sí, sobre escribiendo las funcionalidades de la clase exception.
- ☐ No.

1.3.- En C++, si se define el constructor de una clase como private...

- ☐ Se evita la herencia de dicha clase.
- ☐ En C++ no se puede definir el constructor de una clase como private.
- ☐ No se puede crear objetos de dicha clase.
- ☐ No se puede usar polimorfismo.

1.4.- En C++, si se define un método virtual puro dentro de una clase:

- ☐ No se podrá crear objetos de dicha clase.
- ☐ Se producirá un error de ejecución al intentar instanciar la clase.
- ☐ Se evita la herencia de dicha clase.
- ☐ No se podrá redefinir el método en clases heredadas.

1.5.-Cuál de las siguientes afirmaciones sobre búsqueda adaptativa traspuesta no es cierta:

- ☐ Los elementos de la secuencia se van poco a poco ordenando según su frecuencia de búsqueda
- ☐ Tras cada búsqueda siempre se cambian dos elementos de posición
- ☐ Los elementos de la secuencia se van ordenando de más buscado a menos buscado
- ☐ La implementación es más adecuada usando un vector que una lista enlazada

1.6.- ¿Cuándo se utiliza una técnica de exploración en una tabla de dispersión?

- ☐ Cuando no hay colisión ni desbordamiento
- ☐ Cuando no hay colisión pero hay desbordamiento
- ☐ Cuando hay colisión y desbordamiento
- ☐ Cuando hay colisión pero no hay desbordamiento

1.7.- Las dos subsecuencias en las que el QuickSort divide la secuencia para ordenarlas:

- ☐ Son siempre del mismo tamaño
- ☐ Son siempre de diferente tamaño
- ☐ La diferencia entre sus tamaños es como máximo 1
- ☐ La diferencia entre sus tamaños depende del pivote utilizado

1.8.- Es un método de ordenación por intercambio:

- ☐ HeapSort
- ☐ QuickSort
- ☐ ShellSort
- ☐ ShakeSort

1.9.- La profundidad máxima de un árbol binario de n nodos es:

- ☐ $\log_2 n$
- ☐ $\log_2 n - 1$
- ☐ n
- ☐ $\log_2 n + 1$

1.10.- Dado un árbol binario de búsqueda de números enteros que se inserta en este orden: 5, 15, 3, 4, 12. Indique qué opción corresponde al recorrido en preorden:

- ☐ 3 4 5 12 15
- ☐ 5 3 4 15 12
- ☐ 5 3 15 4 12
- ☐ 4 3 12 15 5

2. Preguntas cortas (cada pregunta es de 1 punto; desarrolle cada una en hoja separada)

2.1.- La colocación de los 9 primeros elementos insertados en una tabla hash con celdas de 3 registros usando el número del DNI sin letra como clave es la siguiente:

0	1	2	3	4	5	6
MILA		SERGIO	MARISA		PACO	JAVI
TERE		VIDAL				DANI
LINO						

Determinar la colocación en la tabla de los elementos que viene a continuación al introducirlos en el orden en el que aparecen aquí, usando una función de dispersión basada en la suma de los dos dígitos centrales de la clave y exploración cuadrática:

VANESA:	44137052
DARA:	57381032
ISA:	43163014
LOLA:	54342304
CARLA:	44011927
BRIAN:	47194036

2.2.- Indica y justifica cuál es el resultado que aparece en pantalla.

```
#include <iostream>

class A {
private:
    int a_;
public:
    A(int x=10): a_(x) {}
    int f(int x) { return g(x * x); }
    virtual int g(int x) const { return x % a_; }
};

class B: public A {
private:
    int b_;
public:
    B(int x): b_(x) {}
    virtual int f(int x) const { return g(x + x); }
    int g(int x) const { return x % b_; }
};

class C: public B {
private:
    int c_;
public:
    C(int x=5): B(x), c_(x) {}
    int f(int x) { return g(x * c_); }
};

int main()
{
    A *a = new A;
    std::cout << a->f(5) << std::endl;

    B *b = new B(7);
    std::cout << b->f(4) << std::endl;

    B *c = new C;
    std::cout << c->f(3) << std::endl;

    return 0;
}
```

2.3.- Describir muy brevemente el método de ordenación de la Sacudida o ShakeSort y aplicarlo para ordenar de **mayor a menor** la secuencia [28, 40, 22, 37, 45, 10, 18, 35, 15]. Debes indicar en cada una de las iteraciones, al menos, el estado de la secuencia, resaltando de manera clara los elementos que se mueven y los que ya se encuentran ordenados.

2.4.- En un árbol AVL, inserta los siguientes valores (por este orden) 25, 15, 20, 30, 23, 27, 22 y luego elimina los 3 nodos de menor valor. Indicar el tipo de rotaciones que se producen y dónde, dibujando el árbol tal como quedaría tras cada una de las operaciones.

Tras finalizar todas las operaciones, escribir el recorrido preorden del árbol resultante.

3. Preguntas de desarrollo (cada pregunta es de 2 puntos; desarrolle cada una en hoja separada)

✓3.1- Escribe una función en C++ que aplique el procedimiento de ordenación siguiente: se realiza una iteración del método Quicksort para dividir la secuencia en dos partes y se ordenan, sobre la propia secuencia, la primera parte por **Selección** y la segunda por **Inserción** en ambos casos por la **derecha**, en lugar de por la izquierda.

N 3.2.- Diseñar e implementar una jerarquía de clases en C++ para representar las piezas del juego del ajedrez. Se trata de un juego de estrategia entre dos jugadores que consta de los siguientes elementos:

- El **tablero** está dividido en 64 casillas, con distribución 8x8, alternando casillas blancas y negras. Las ocho líneas verticales se denominan columnas, y se identifican con las letras de la 'a' a la 'h'; las ocho líneas horizontales se denominan filas, y se identifican con los números del 1 al 8. Cada casilla se identifica por la pareja (letra de columna, número de fila).
- Hay seis tipos de **piezas**: **Rey**, **Dama** (o reina), **Torre**, **Alfil**, **Caballo** y **Peón**. Al iniciar la partida cada jugador dispone de 16 piezas: 1 rey, 1 dama, 2 torres, 2 alfiles, 2 caballos y 8 peones. Para diferenciar las piezas de cada jugador, las de un jugador son **blancas** y las del otro son **negras**. Inicialmente las piezas blancas se disponen en las filas 1 y 2; y las piezas negras en las filas 7 y 8. Cada pieza conoce su color y la posición que ocupa en el tablero. En cada turno de juego un jugador **mueve** una de sus piezas. En general todas las piezas siguen las siguientes reglas de movimiento:
 - Una pieza se mueve a una casilla vacía. No puede ocupar una casilla ocupada por otra pieza del mismo color, pero sí una casilla ocupada por una pieza del oponente, retirándola del tablero (captura).
 - Las piezas no pueden saltar, en su movimiento, por encima de otras piezas, salvo en el movimiento del caballo.

La implementación del movimiento de una pieza se realiza mediante un método virtual en la clase base **Pieza** que recibe por parámetro la casilla (columna, fila) destino. Este método es responsable de calcular si el movimiento es posible en función del color y de las reglas de movimiento propias de cada tipo de pieza. A continuación se indican las reglas básicas de movimiento para cada tipo de pieza que se pide implementar:

- El **Peón** se mueve hacia adelante una casilla, si esta está vacía. No puede moverse hacia atrás. Puede capturar una pieza adversaria en cualquiera de las casillas diagonales en frente del peón.
- El **Alfil** se mueve en direcciones diagonales sobre casillas del mismo color. Se puede mover tantas casillas como desee, pero sólo en una dirección en cada turno.
- Se pide especificar la clase que define al **Rey**, pero no se implementará el código para su operación de movimiento. En caso de invocarse la operación movimiento sobre esta pieza se lanzará una excepción de tipo `Not_yet_implemented` que deriva de la clase `std::exception`.
- La operación movimiento de una pieza se implementa mediante un método que recibe como parámetro

Se pide escribir un programa principal que declare tres punteros a piezas inicializados con:

```
p1 <- Peon(Blanco, (2,d))
p2 <- Rey(Blanco, (1,d))
p3 <- Alfil(Negro, (8,c))
```

A continuación se realizan los siguientes movimientos, que estarán dentro de un bloque para detectar y manejar las posibles excepciones.

```
p1->mover(3,d); p2->mover(2,d); p3->mover(7,c);
```